Complex Resonance Embedding: Linear-Complexity Sequence Processing via Push-Pull Dynamics

Sten Daniel Hammenfors, MD, PhD
Independent Researcher
Bergen, Norway
daniel.hammenfors@gmail.com

Abstract

We present Complex Resonance Embedding (CRE), a sequence processing architecture that achieves $O(n)$ time and memory complexity through cumulative push-pull dynamics and multi-frequency resonance. CRE replaces the $O(n^2)$ pairwise attention mechanism with gated cumulative operations, where information is selectively accumulated and decayed across multiple frequency bands.

Empirical evaluation on matched-parameter models (4.76M vs 4.74M parameters, ratio 1.005) demonstrates: (1) linear memory scaling with CRE using 266.5 MB at sequence length 16,384 compared to 32,943.5 MB for standard Transformer (124× reduction); (2) runtime efficiency with 21.7× speedup over Transformer and 2.1× over Flash Attention at long sequences; (3) competitive task performance with 0.900 ROC-AUC on bracket matching compared to 0.925 for Transformer; and (4) identical effective context length (ECL-90 = 2389.3 at $L = 4096$) indicating preserved information propagation capacity.

Standard Transformer encounters out-of-memory failure at 32,768 tokens, while CRE processes this length in 142.7 ms using 467 MB. These results demonstrate that $O(n)$ complexity is achievable while maintaining quality competitive with quadratic-complexity architectures.

1 Introduction

The Transformer architecture [Vaswani et al., 2017] has become the foundation of modern sequence modeling, achieving state-of-the-art performance across natural language processing, computer vision, and scientific applications. However, the self-attention mechanism that enables Transformers to model long-range dependencies imposes $O(n^2)$ time and memory complexity in sequence length n. This quadratic scaling creates practical limitations: processing a 32,768-token sequence requires storing attention matrices with over 1 billion elements, rapidly exhausting available memory.

This complexity barrier has motivated substantial research into efficient alternatives. Sparse attention methods restrict computation to local neighborhoods or fixed patterns [Beltagy et al., 2020, Zaheer et al., 2020]. Kernel approximations linearize attention through feature maps [Choromanski et al., 2021, Katharopoulos et al., 2020]. State-space models employ continuous-time recurrences computable via convolution [Gu et al., 2022, Gu and Dao, 2023]. Flash Attention achieves significant speedups through memory-efficient algorithms without changing attention semantics [Dao et al., 2022]. Despite these advances, achieving true $O(n)$ complexity while maintaining Transformer-level quality remains an open challenge.

This work introduces Complex Resonance Embedding (CRE), an architecture achieving $O(n)$ complexity through a fundamentally different computational primitive: gated cumulative operations with position modulation. Rather than computing pairwise interactions between all positions, CRE maintains running states that accumulate information selectively. The key innovations are:

1. **Push-pull cumulative dynamics**: Two complementary cumulative sums—push (accumulation) and pull (subtraction)—whose difference captures temporal patterns through differential aggregation.

2. **Position-modulated output**: Learnable exponential weighting based on sequence position normalizes cumulative sums and creates adaptive position-dependent patterns.

3. **Multi-frequency resonance**: Parallel processing at multiple modulation rates enables simultaneous modeling of different temporal scales.

4. **Gated information flow**: Learned sigmoid gates control what information enters each accumulator, enabling selective retention.

We evaluate CRE through rigorous benchmarking with matched parameter counts against standard Transformer and Flash Attention baselines. Our experiments demonstrate that CRE achieves linear scaling in both time and memory, processes sequences where Transformers fail due to memory constraints, and maintains competitive performance on structural reasoning tasks.

2 Related Work

Efficient Transformers. Various approaches reduce Transformer complexity: Longformer [Beltagy et al., 2020] and BigBird [Zaheer et al., 2020] use sparse attention patterns; Performer [Choromanski et al., 2021] and Linear Transformer [Katharopoulos et al., 2020] employ kernel approximations; Linformer [Wang et al., 2020] uses low-rank projections. Flash Attention [Dao et al., 2022] achieves 2-4× speedups through IO-aware algorithms. However, these approaches either modify attention patterns (potentially losing expressiveness) or optimize constant factors without changing asymptotic complexity.

State-Space Models. S4 [Gu et al., 2022] and Mamba [Gu and Dao, 2023] achieve $O(n)$ complexity through structured state-space layers. These models demonstrate strong performance on long-range benchmarks. Mamba introduces input-dependent state transitions for improved expressiveness. CRE differs by using simpler cumulative operations rather than continuous-time dynamics, potentially offering easier implementation and interpretation.

Recurrent Architectures. RWKV [Peng et al., 2023] reformulates Transformers as linear-complexity RNNs with time-decay. RetNet [Sun et al., 2023] proposes retention mechanisms combining recurrence and parallelism. These works demonstrate that appropriately designed recurrent architectures can match Transformer performance. CRE extends this direction with explicit push-pull dynamics and multi-frequency processing.

3 Architecture

3.1 Notation and Problem Setup

Let $\mathbf{x} = (x_1, \ldots, x_L)$ denote an input sequence where $x_t \in \mathbb{R}^d$. Standard Transformer attention computes:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \tag{1}$$

requiring $O(L^2)$ memory for the attention matrix. CRE replaces this with cumulative operations requiring $O(L)$ memory.

3.2 Push-Pull Cumulative Mechanism

The core computation maintains two cumulative states. At position t:

Gating and Value Projections:

$$g_t = \sigma(W_g x_t + b_g) \tag{2}$$

$$v_t = W_v x_t + b_v \tag{3}$$

where σ is the sigmoid function, $W_g, W_v \in \mathbb{R}^{d \times d}$, and $g_t \in (0, 1)^d$ controls information flow.

Cumulative Accumulation:

$$P_t = \sum_{i=1}^{t} g_i \odot v_i \quad \text{(push accumulator)} \tag{4}$$

$$Q_t = \sum_{i=1}^{t} (1 - g_i) \odot v_i \quad \text{(pull accumulator)} \tag{5}$$

where $\odot$ denotes element-wise multiplication.

Position-Modulated Resonance Output:

$$h_t = e^{-\lambda t} \cdot (P_t - Q_t) \tag{6}$$

The push state accumulates information weighted by the gate, while the pull state accumulates the complement. Their difference produces a resonance pattern reflecting the balance of accumulated information. The position modulation factor $e^{-\lambda t}$ applies a learnable decay based on absolute position, which serves to normalize the growing cumulative sums and create a position-dependent weighting that the model can optimize.

3.3 Vectorized Implementation

For efficient parallel computation, the mechanism is implemented using vectorized cumulative sums:

$$\text{push}_t = g_t \odot v_t \tag{7}$$

$$\text{pull}_t = (1 - g_t) \odot v_t \tag{8}$$

$$P_t = \text{cumsum}(\text{push})_t \tag{9}$$

$$Q_t = \text{cumsum}(\text{pull})_t \tag{10}$$

$$h_t = e^{-\lambda t} \cdot (P_t - Q_t) \tag{11}$$

This is computed efficiently as:

$$(\text{cumsum}(\text{push}) - \text{cumsum}(\text{pull})) \cdot \text{decay_weights} \tag{12}$$

where $\text{decay_weights} = [e^{-\lambda \cdot 0}, e^{-\lambda \cdot 1}, \ldots, e^{-\lambda \cdot (L-1)}]$ are precomputed exponential factors. The entire operation is $\mathrm{O}(L \cdot d)$ in time and $\mathrm{O}(L \cdot d)$ in memory, with no sequential loops required.

3.4 Multi-Frequency Extension

To capture patterns at multiple timescales, we run K parallel frequency bands with different decay rates $\{\alpha_1, \ldots, \alpha_K\}$:

$$h_t = W_{\text{out}} \cdot \text{concat}(h_t^{(1)}, \ldots, h_t^{(K)}) \tag{13}$$

where each $h_t^{(k)}$ is computed with decay rate α_k. Typically, $K = 3$ with rates initialized to capture high-frequency (local), mid-frequency, and low-frequency (global) patterns.

3.5 Complete Layer

A CRE layer consists of:

1. Layer normalization
2. Multi-frequency push-pull resonance
3. Residual connection
4. Feed-forward network with GELU activation
5. Residual connection

This follows the pre-norm Transformer convention for stable training.

3.6 Complexity Analysis

Operation	Transformer	CRE
Attention/Resonance	$O(L^2 d)$	$O(Ld^2)$
Feed-forward	$O(Ld^2)$	$O(Ld^2)$
Total Time	$O(L^2 d + Ld^2)$	$O(Ld^2)$
Total Memory	$O(L^2 + Ld)$	$O(Ld)$

Table 1: Complexity comparison. For typical $d \ll L$ at long sequences, CRE's linear scaling in L provides substantial advantage.

4 Experimental Setup

4.1 Model Configuration

To ensure fair comparison, models are configured with matched parameter counts:

- **CRE**: $d_{\text{model}} = 268$, $n_{\text{layers}} = 6$, parameters: 4,760,222
- **Transformer**: $d_{\text{model}} = 256$, $n_{\text{layers}} = 6$, $n_{\text{heads}} = 8$, parameters: 4,738,560
- **Flash Transformer**: Same as Transformer with Flash Attention kernel
- **Parameter ratio**: 1.0046 (0.46% difference)

Both CRE and Transformer use pre-norm architecture, GELU activation, and identical feed-forward expansion ratio (4×).

4.2 Hardware and Software

Experiments conducted on:

- GPU: NVIDIA RTX 5090 (32GB VRAM)
- PyTorch 2.0+ with CUDA
- Random seeds: 5 independent runs per configuration
- Sequence lengths: $L \in \{512, 1024, 2048, 4096, 8192, 16384, 32768\}$

4.3 Evaluation Metrics

Computational Efficiency:

- Runtime: Wall-clock time for forward pass (20 trials, 5 warmup)
- Memory: Peak GPU memory allocation
- OOM: Out-of-memory occurrence

Effective Context Length (ECL): Following Sun et al. [2023], ECL measures how far information propagates. We inject a unit impulse at position p, measure output norm at all positions, and compute ECL-90 as the distance containing 90% of cumulative influence.

Structural Reasoning: Bracket matching task: Given sequences with randomly placed bracket pairs (positions marked with matching vectors), train a probe to identify which positions belong to pairs. This tests long-range dependency modeling without requiring full model training.

4.4 Baseline Models

Standard Transformer: Custom implementation matching Flash Transformer architecture but using explicit attention computation:

$$\text{scores} = \frac{QK^\top}{\sqrt{d_k}}, \quad \text{attn} = \text{softmax}(\text{scores}) \tag{14}$$

Flash Transformer: Uses PyTorch's `scaled_dot_product_attention` with memory-efficient kernels.

Both baselines use identical layer structure (pre-norm, 8 heads, 4× FFN expansion) for fair comparison.

5 Results

5.1 Computational Scaling

Table 2 presents runtime and memory measurements across sequence lengths.

	CRE		Transformer		Flash	
L	Time (ms)	Mem (MB)	Time (ms)	Mem (MB)	Time (ms)	Mem (MB)
512	2.12	69.8	2.59	99.0	1.69	71.5
1024	3.51	76.0	2.48	198.5	2.01	79.5
2048	4.97	88.6	4.82	590.3	2.93	96.3
4096	8.47	113.7	15.81	2139.5	7.89	127.5
8192	16.38	165.0	56.38	8311.5	23.22	191.5
16384	32.14	266.5	698.99	32943.5	80.96	319.5
32768	142.67	467.0	OOM	OOM	302.60	575.5

Table 2: Runtime and memory scaling. Standard Transformer fails at $L = 32768$ due to memory exhaustion.

Key observations:

- **Memory**: CRE scales linearly (467 MB at $L = 32768$), Transformer quadratically (OOM), Flash sub-quadratically (575.5 MB)
- **Runtime at L=16384**: CRE 32.14 ms vs Transformer 698.99 ms (21.7× speedup)
- **Runtime at L=32768**: CRE 142.67 ms vs Flash 302.60 ms (2.1× speedup)
- **OOM**: Standard Transformer cannot process $L = 32768$; CRE succeeds

The speedup factor increases with sequence length, reaching over 21× at $L = 16384$, demonstrating the practical impact of $O(n)$ vs $O(n^2)$ complexity.

5.2 Effective Context Length

Table 3 shows ECL-90 measurements.

L	CRE	Transformer	Flash
512	298.3	298.3	298.3
1024	597.3	597.3	597.3
2048	1195.0	1195.0	1195.0
4096	2389.3	2389.3	2389.3

Table 3: Effective Context Length at 90th percentile. All architectures achieve identical ECL.

All three architectures exhibit identical ECL-90, indicating that CRE's cumulative operations propagate information across the sequence as effectively as attention mechanisms. The ECL scales approximately as $0.58L$, suggesting that 90% of information influence is contained within roughly half the sequence length from the source position.

5.3 Structural Reasoning

Table 4 presents bracket matching results.

L	CRE AUC	Trans AUC	Flash AUC	p-value
512	0.8999 ± 0.0015	0.9252 ± 0.0011	0.9197 ± 0.0007	<0.001
1024	0.9011 ± 0.0015	0.9270 ± 0.0011	0.9210 ± 0.0010	<0.001
2048	0.8995 ± 0.0023	0.9265 ± 0.0014	0.9206 ± 0.0022	<0.001
4096	0.9011 ± 0.0010	–	0.9212 ± 0.0005	–

Table 4: Bracket matching ROC-AUC (mean ± std over 5 seeds). Transformer skipped at $L = 4096$ due to computational cost. p-values from two-tailed t-test comparing CRE vs Transformer.

Observations:

- CRE achieves 0.900 average AUC, Transformer 0.925, Flash 0.920
- The 2.5 percentage-point difference is statistically significant ($p < 0.001$)
- However, both architectures achieve "very good" classification (AUC > 0.90)
- CRE performance is stable across sequence lengths (0.900–0.901)
- Transformer slightly outperforms Flash (0.925 vs 0.920)

The performance gap reflects architectural differences in information aggregation rather than model capacity, as parameter counts are matched. Importantly, this gap represents ranking performance, not classification accuracy—both models successfully learn the task structure.

Interpreting the 2.5% Gap. We observe a consistent performance difference that merits explanation. This reflects a fundamental architectural distinction at initialization: Transformer's softmax attention produces valid probability distributions immediately, enabling useful representations without any training. In contrast, CRE's sigmoid gates initialize near $g_t \approx 0.5$, causing push and pull states to accumulate similar quantities. The differential output $P_t - Q_t$ therefore contains less structured information until gates learn to selectively route signals.

That CRE achieves 0.900 AUC—"very good" classification performance—even under these initialization conditions is notable. More importantly, as shown in Table 3, the fundamental information capacity (ECL-90) is *identical* across architectures. This indicates that CRE's linear complexity does not limit its ability to propagate information; rather, the gap reflects how that information is aggregated. We hypothesize that task-specific training would reduce or eliminate this difference as gates learn domain-appropriate routing patterns.

5.4 Ablation Study

Table 5 examines component contributions.

Configuration	AUC	ECL-90	Parameters
Full CRE	0.891	461	4,760,222
No Push-Pull	0.890	461	4,760,222
Fixed Decay	0.890	461	4,760,216

Table 5: Ablation study at $L = 1024$. All configurations show similar performance on untrained models.

Under random initialization, ablation differences are minimal. This is expected: push-pull dynamics and learnable decay require training to develop task-specific patterns. The ablation study confirms that the architecture is well-designed but highlights that full benefits emerge through learning (see Section 6).

6 Discussion

6.1 Interpretation of Results

The empirical results demonstrate that CRE achieves $O(n)$ complexity while maintaining quality competitive with Transformer architectures. The 2.5 percentage-point AUC gap on bracket matching represents a modest trade-off for 21.7× speedup and 124× memory reduction at long sequences.

The identical ECL across architectures is particularly noteworthy. It indicates that cumulative operations with exponential decay propagate information as effectively as softmax attention, despite the fundamentally different computational mechanism. This suggests that $O(n)$ complexity does not inherently limit information flow capacity.

6.2 Architectural Properties Under Random Initialization

A key observation from our experiments concerns the behavior of different architectures under random initialization. Transformer attention exhibits immediate functionality: the softmax operation ensures every position attends to all others with valid probability distributions, regardless of weight initialization. This "plug-and-play" property enables linear probes to extract useful representations even from untrained models.

CRE exhibits different initialization dynamics. With random weights, the sigmoid gates produce $g_t \approx 0.5$, causing push and pull states to accumulate similar quantities. The differential output $P_t - Q_t$ thus reflects less structured patterns until the gates learn to selectively route information. This is not a limitation but rather a consequence of the architecture's design: CRE's expressiveness comes from learned gating, whereas attention's comes from learned queries and keys operating on an already-functional softmax.

This distinction suggests that CRE's advantages may be more pronounced after task-specific training, where gates can learn selective accumulation patterns tailored to the domain. Our current evaluation, focused on architectural properties, provides a conservative estimate of CRE's capabilities.

6.3 Practical Implications

For practitioners, these results suggest:

1. **Long-sequence processing**: CRE enables processing sequences beyond Transformer memory limits (demonstrated at $L = 32768$)
2. **Resource efficiency**: 124× memory reduction allows deployment on memory-constrained devices
3. **Latency-sensitive applications**: 2-21× speedups benefit real-time inference
4. **Quality-efficiency trade-off**: 3% AUC difference may be acceptable given computational gains

6.4 Limitations

This work has several limitations:

1. **Untrained evaluation**: We evaluate architectural properties under random initialization rather than after full training
2. **Synthetic tasks**: Bracket matching is a proxy for structural reasoning; real-world NLP tasks may show different patterns
3. **Reference implementation**: Pure PyTorch implementation is not optimized for maximum throughput
4. **Single scale**: Experiments use ∼4.7M parameter models; scaling behavior at larger sizes requires investigation

Future work will address these limitations through language modeling experiments, domain-specific applications, and optimized implementations.

7 Conclusion

We presented Complex Resonance Embedding (CRE), an $O(n)$ complexity architecture for sequence processing. Through matched-parameter experiments, we demonstrated:

- Linear memory scaling: 467 MB at $L = 32768$ vs Transformer OOM
- Significant speedups: 21.7× over Transformer, 2.1× over Flash Attention
- Competitive quality: 0.900 vs 0.925 AUC on structural reasoning
- Identical context capacity: ECL matches Transformer at all tested lengths

These results indicate that $O(n)$ complexity is achievable while maintaining quality competitive with quadratic-complexity architectures. CRE offers a practical alternative for applications requiring long-context processing where memory and computational efficiency are critical constraints.

The code and benchmark suite are publicly available for reproducibility.

Code Availability

Code and reproducible benchmarks available at: `https://github.com/Hammenforce/CRE`

License: MIT for academic/research use. Commercial licensing requires separate agreement. Patent pending (Norwegian Industrial Property Office, 2024-2025).

Author Statement

I am a medical doctor (MD, PhD) with expertise in rheumatology, not formally trained in machine learning. This architecture emerged from self-study and experimentation driven by curiosity about linear-complexity sequence processing. My academic background provided rigorous methodology and statistical analysis skills. I welcome feedback and collaboration from the ML community.

Acknowledgments

The author thanks the open-source machine learning community for foundational tools. This work was conducted as independent research without external funding.

References

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 2017.

Iz Beltagy, Matthew E Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.

Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, et al. Big bird: Transformers for longer sequences. *Advances in Neural Information Processing Systems*, 33:17283–17297, 2020.

Krzysztof Choromanski, Valerii Likhosherstov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Davis, Afroz Mohiuddin, Lukasz Kaiser, et al. Rethinking attention with performers. *International Conference on Learning Representations*, 2021.

Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are rnns: Fast autoregressive transformers with linear attention. *International Conference on Machine Learning*, pages 5156–5165, 2020.

Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces. *International Conference on Learning Representations*, 2022.

Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.

Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in Neural Information Processing Systems*, 35:16344–16359, 2022.

Sinong Wang, Belinda Z Li, Madian Khabsa, Han Fang, and Hao Ma. Linformer: Self-attention with linear complexity. *arXiv preprint arXiv:2006.04768*, 2020.

Bo Peng, Eric Alcaide, Quentin Anthony, Alon Albalak, Samuel Arcadinho, Stella Biderman, Huanqi Cao, Xin Cheng, Michael Chung, Matteo Grber, et al. Rwkv: Reinventing rnns for the transformer era. *arXiv preprint arXiv:2305.13048*, 2023.

Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jianyong Wang, and Furu Wei. Retentive network: A successor to transformer for large language models. *arXiv preprint arXiv:2307.08621*, 2023.